Uvod u softversko inženjerstvo
Avalonia UI framework

Fakultet tehničkih nauka
Univerzitet u Novom Sadu

Kako koristiti materijal

Materijal je organizovan kao mini-aplikacije (po temi). Za svaku temu napravite novi Avalonia projekat u posebnom folderu i zatim copy/paste kod iz odgovarajuće sekcije.

Ne pravimo zajednički solution. Svaki folder je zaseban projekat koji otvarate direktno u Rider-u (File/Open).

Preporuka: koristite isto ime projekta (`GetStartedApp`) u svim mini-projektima, da ne biste menjali `x:Class` i namespace u snippeti-ma.

- **01-xaml** – osnove XAML-a i kontrole
- **02-events** – događaji i event handler-i
- **03-layouts** – paneli i raspored
- **04-binding** – data binding, resursi, konverzija

Kreiranje mini-projekta (CLI primer):

```
dotnet new install Avalonia.Templates
dotnet new avalonia.app -o 03-layouts -n GetStartedApp
```

Contents

1 Uvod u Avalonia **3**
1.1 Kreiranje Avalonia projekta (Rider) . . . 3
1.2 Markup i code-behind . . . 5

2 Markup (XAML) **7**

3 Code-behind **10**

4 Događaji i obrada događaja (eng. Events and handling events) **11**
4.1 Dodavanje event handler-a u Avalonia (Rider) . . . 11

5 Paneli (eng. Panels) **13**
5.1 Wrap panel . . . 14
5.2 Stack panel . . . 15
5.3 Grid panel . . . 16
5.4 Dock panel . . . 17
5.5 Primer kombinacije više panela – Complex Layout . . . 19

6 Data binding **21**
6.1 Priprema: DataContext (primer) 22
6.2 Binding na resurs – string (1) 24
6.3 Binding na resurs – boja (2) 24
6.4 One way – ka interfejsu (7) 24
6.5 Two way – bidirekciono (6) 25
6.6 One way to source – ka kodu (8) 25
6.7 Binding kontrole na kontrolu (3) 25
6.8 Binding kontrole na kontrolu - real time (4) 25
6.9 Binding kontrole na kontrolu - real time + konverzija (5) 26
6.10 Binding na listu (9) 27

1 Uvod u Avalonia

Avalonia UI je cross-platform GUI (Graphical User Interface) framework za kreiranje desktop aplikacija na Windows-u, Linux-u i macOS-u.

GUI framework omogućava da kreiramo aplikacije uz pomoć velikog opsega GUI elemenata kao što su labele (eng. labels) i polja za unos teksta (eng. textboxes). Bez GUI framework-a, morali bi da crtamo ove elemente ručno i da upravljamo sa svim korisničkim interakcijama, kao što je unos teksta ili pomeranje miša.

Avalonia sadrži set application-development alata (eng. features): XAML, kontrole, data binding, layouts, 2D grafika, animacije, stilovi, media i slično.

1.1 Kreiranje Avalonia projekta (Rider)

Preduslovi:

- .NET SDK (10.0 LTS)
- JetBrains Rider
- (Opcionalno) Rider plugin *AvaloniaRider* za live XAML preview

Instalacija Avalonia templates:

```
dotnet new list
dotnet new install Avalonia.Templates
```

Kreiranje projekta u Rider-u:

1. Na početnom ekranu izaberite **New Solution** (Rider-ov naziv za kreiranje novog projekta/aplikacije).
2. U sekciji **Custom Templates** izaberite **Avalonia .NET App**.
3. Unesite ime rešenja/projekta i lokaciju.
4. Kliknite **Create**.

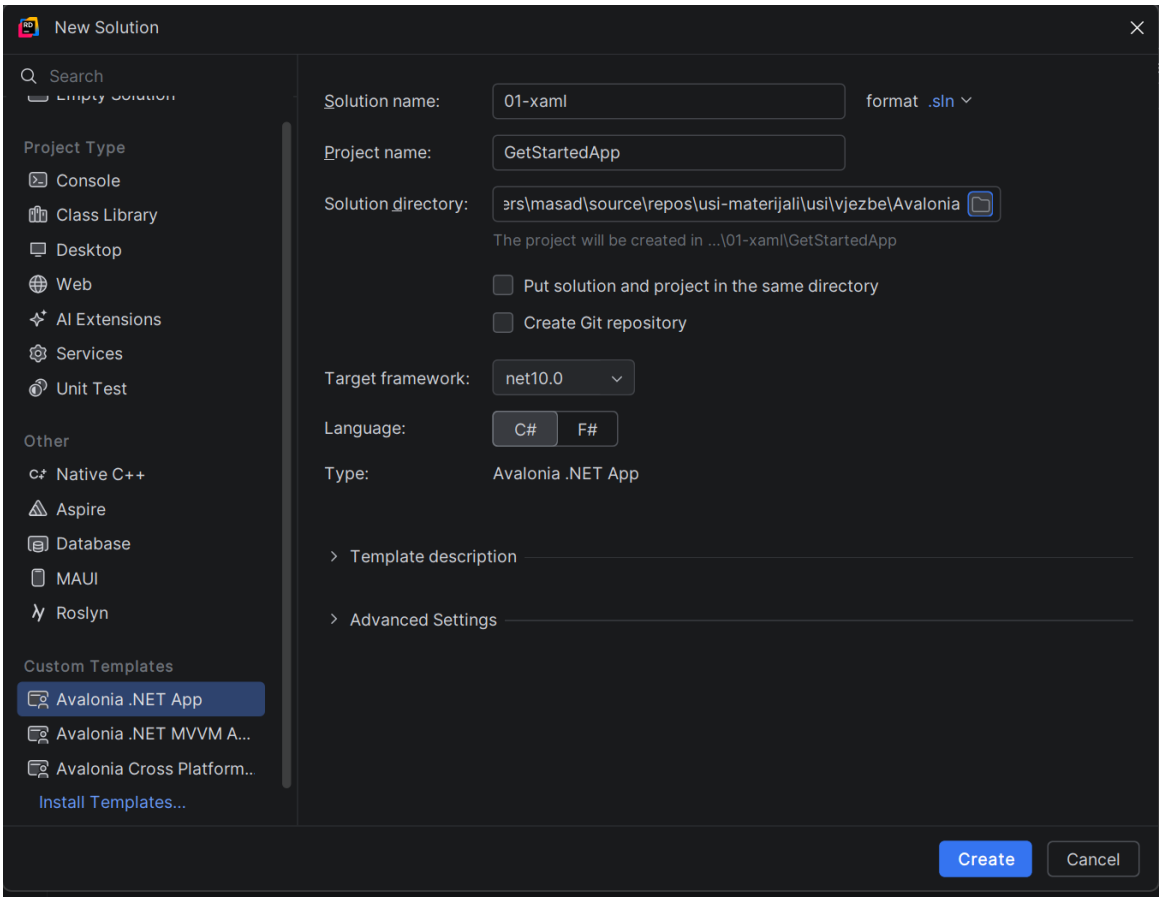

Figure 1: Slika 1. Kreiranje novog Avalonia projekta u Rider-u

Alternativa (command line):

```
dotnet new avalonia.app -o 01-xaml -n GetStartedApp
```

Pokretanje projekta:
U Rider-u kliknite **Run**. Alternativno, u terminalu pokrenite `dotnet run` u folderu projekta.

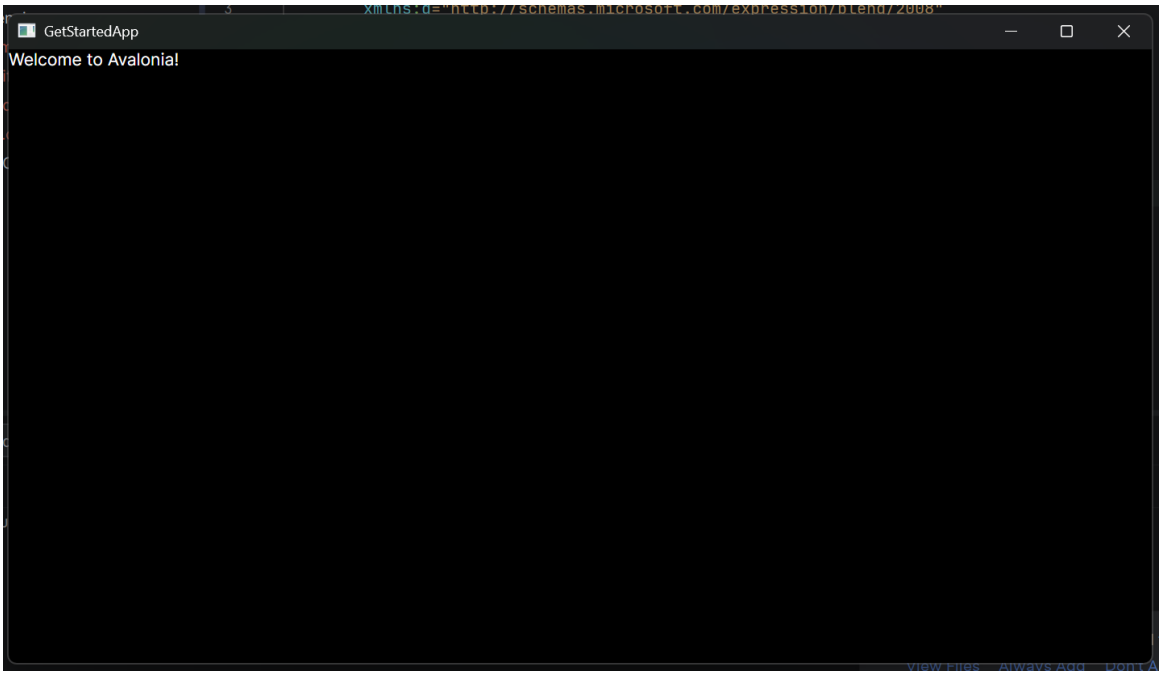

Figure 2: Slika 2. Pokretanje Avalonia aplikacije (podrazumevani ekran)

1.2 Markup i code-behind

Avalonia omogućava da se razvija aplikacija koristeći markup i code-behind.

XAML markup se obično koristi za implementaciju izgleda aplikacije dok se code-behind koristi za implementaciju ponašanja tih komponenti.

Ovo razdvajanje na izgled i ponašanje povlači neke od prednosti kao što su:

1. Cena razvoja i održavanja aplikacije je smanjena jer je znatno lakše upravljati sa aplikacijom kod koje izgled i ponašanje nisu usko vezani (eng. tightly coupled).
2. Razvoj aplikacije je ubrzan i efikasan jer dizajneri mogu implementirati izgled aplikacije nezavisno i paralelno od programera koji implementiraju ponašanje aplikacije.
3. Globalizacija i lokalizacija za desktop aplikacije je pojednostavljena.

Na slici je prikazan prazan Avalonia projekat otvoren u Rider-u. Tipični segmenti rada su:

1. **Solution** prozor (pregled projekta i fajlova).
2. **XAML editor** (npr. `MainWindow.axaml`).
3. (Opcionalno) **XAML preview** uz Rider plugin *AvaloniaRider*.
4. **Code-behind** fajl (npr. `MainWindow.axaml.cs`).

Za razliku od workflow-a sa drag&drop dizajnerom, u Rider-u se kontrole najčešće dodaju pisanjem XAML-a uz code completion i snippet-e.

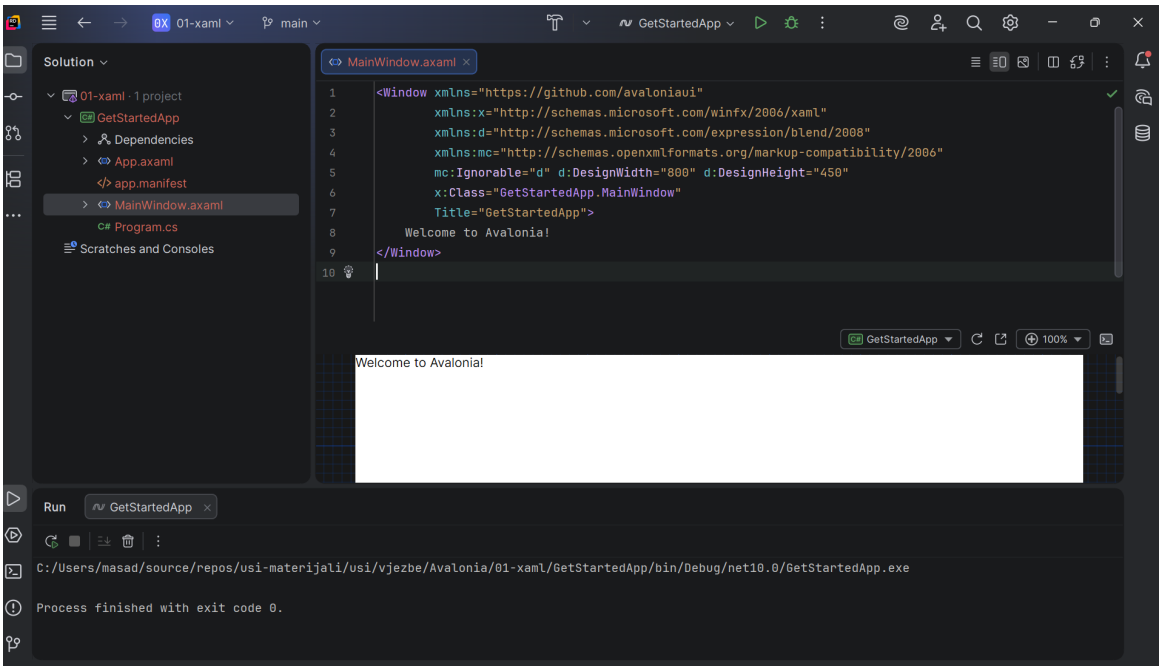

Figure 3: Slika 3. Rider okruženje: Solution, XAML editor i (opciono) XAML preview

Otvaranjem `MainWindow.axaml.cs` dobija se code-behind.

Figure 4: Slika 4. Code-behind (`*.axaml.cs`)

Prelazak između XAML fajla i code-behind fajla vrši se iz **Solution** prozora ili korišćenjem pretrage (Search Everywhere).

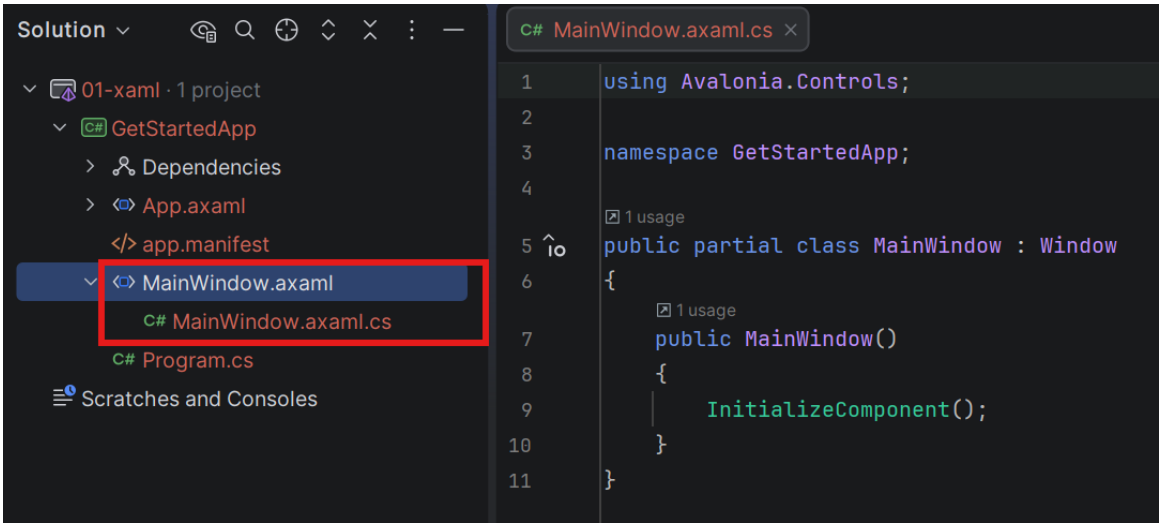

Figure 5: Slika 5. Navigacija: otvaranje `MainWindow.axaml.cs` iz Solution prozora

Ako koristite XAML preview, obično je dostupan kao kartica **Preview** pored XAML editora (uz instaliran *AvaloniaRider* plugin).

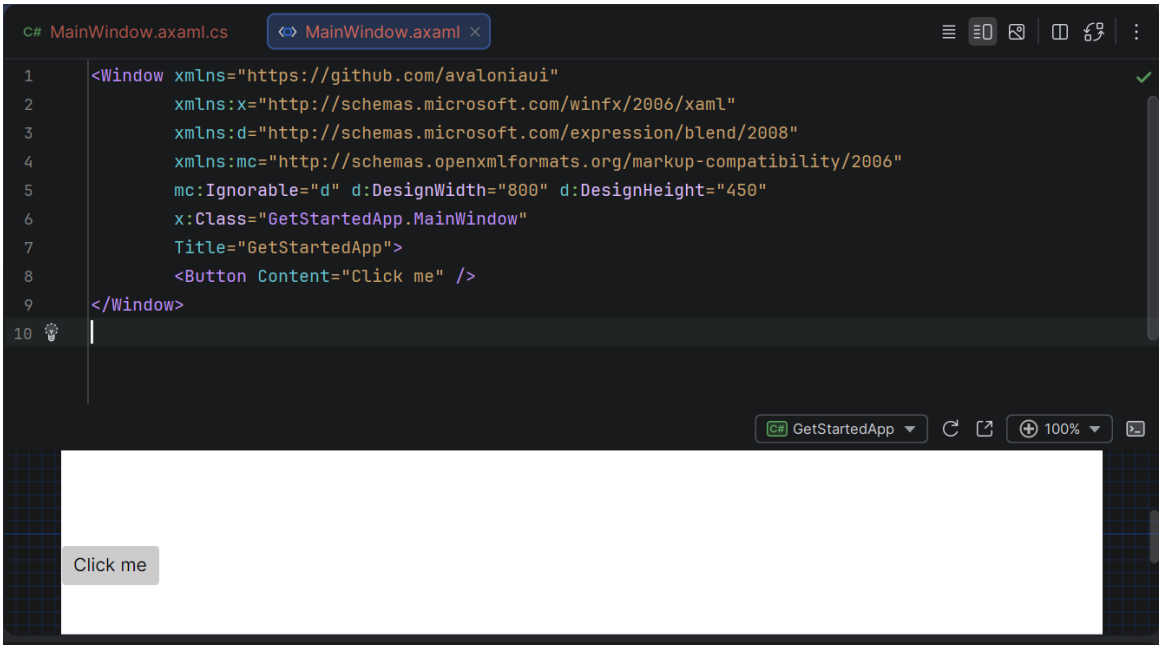

Figure 6: Slika 6. XAML preview u Rider-u (opciono)

2 Markup (XAML)

Mini-projekat: 01-xaml

Pre nego što krenete sa ovom sekcijom preporučeno je da kreirate projekat iz terminala, pa otvorite folder u Rider-u.

```
dotnet new avalonia.app -o 01-xaml -n GetStartedApp
```

U Rider-u otvorite folder `01-xaml` (File/Open) i pokrenite aplikaciju (Run).

Fajl koji menjate: `MainWindow.axaml`

XAML je XML-bazirani markup jezik koji omogućava implementiranje izgleda aplikacije (eng. application's appearance). Koristi se za kreiranje prozora, jednostavnih dijaloga i kontrola a navedeno dalje možemo popunjavati sa novim kontrolama, oblicima i grafičkim elementima.

U Avalonia projektima XAML fajlovi obično imaju ekstenziju `.axaml` (npr. `MainWindow.axaml`).

Primer prikazuje Avalonia XAML kod jednostavnog prozora (tag `<Window>`) koji sadrži samo jedno dugme (tag `<Button>`).

```
<Window xmlns="https://github.com/avaloniaui"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        x:Class="GetStartedApp.MainWindow"
        Title="MainWindow" Width="800" Height="450">
  <Button Content="Click me" />
</Window>
```

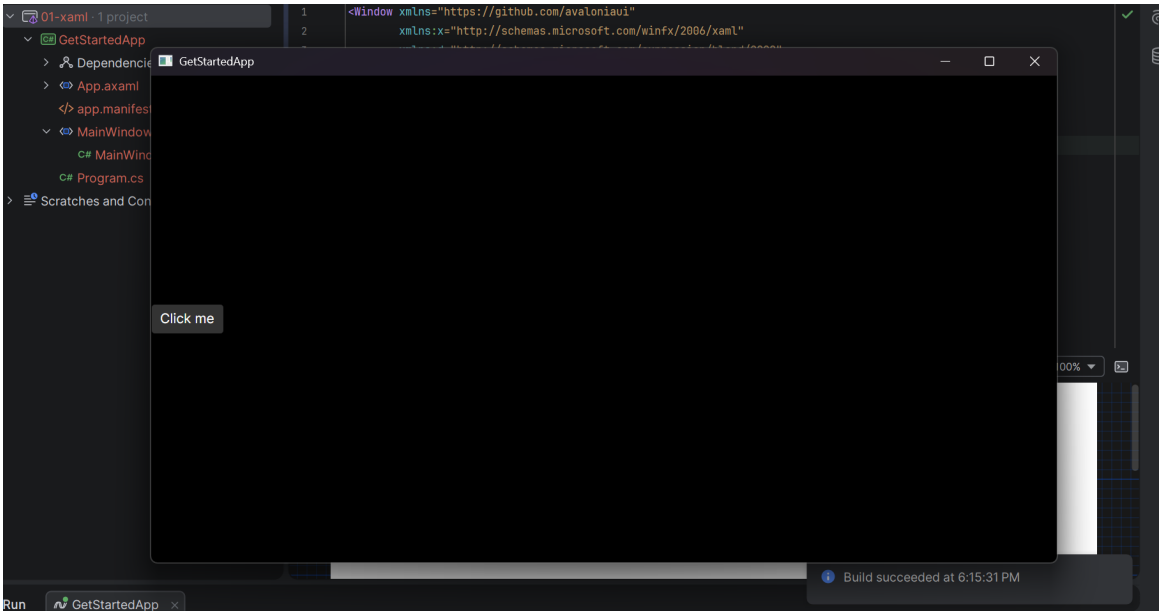

Figure 7: Rezultat 1: Jednostavan Avalonia prozor sa jednim dugmetom

Svaki element može da sadrži određene atribute, slično kao u XML-u. U primeru se vidi da tag `<Window>` sadrži Title atribut koji je zapravo tekst u title baru.

U primeru je prikazano kako se uz pomoć FontWeight property može postaviti boldovan tekst sa sadržajem "A button".

Napomena: sledeći primeri sa `<Button>` su XAML fragmenti. Kopirajte ih u `MainWindow.axaml` unutar `<Window>...</Window>`, kao sadržaj prozora (npr. zamenite postojeće dugme).

```
<Button Content="A button" FontWeight="Bold" />
```

Još jedan način definisanja atributa je prikazan. Ovaj način prikazuje atribute kao child tagove glavnog taga, a u ovom slučaju to je `<Button>`.

```
<Button>
  <Button.Content>A button</Button.Content>
  <Button.FontWeight>Bold</Button.FontWeight>
</Button>
```

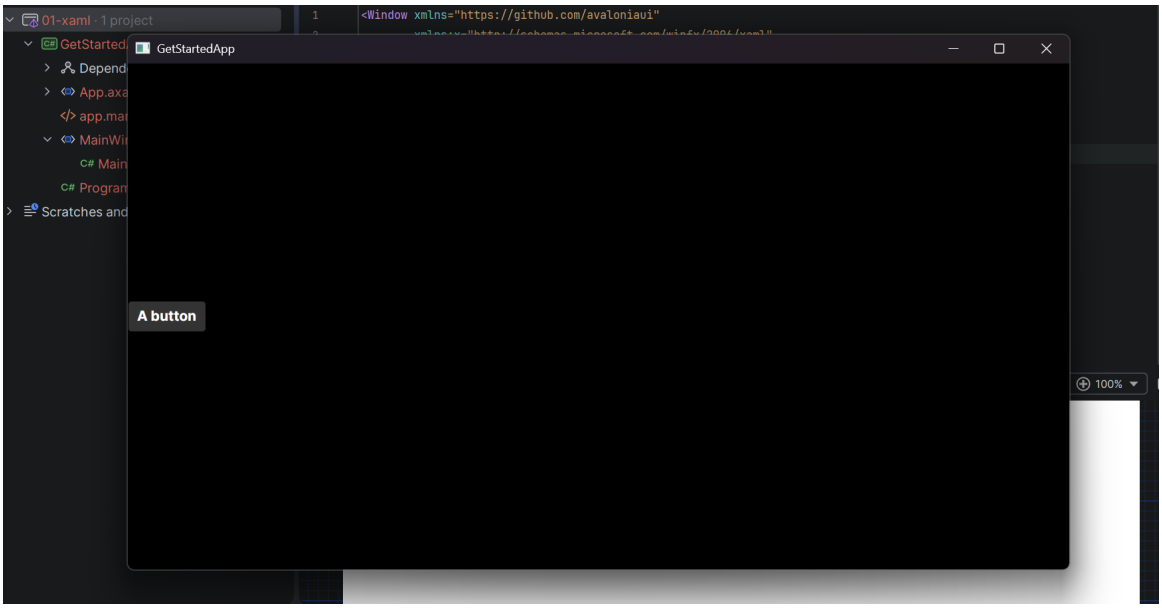

Figure 8: Rezultat 2: Dugme sa boldovanim tekstom (isti vizuelni rezultat za oba prikazana načina pisanja)

S obzirom da je XAML XML-bazirani jezik, UI koji se kreira predstavlja hijerarhiju ugnježdenih elementa poznatih pod nazivom element tree.

Dat je primer gde je u okviru jednog dugmeta kreirano više tekst blokova koristeći više `<TextBlock>` tagova u `<Button>` tagu.

Tekst u tekst blokovima je obojen različitim bojama koristeći atribut Foreground.

```
<Button>
  <Button.Content>
    <WrapPanel>
      <TextBlock Text="A" Foreground="Red" />
      <TextBlock Text=" " />
      <TextBlock Text="multi" Foreground="Green" />
      <TextBlock Text=" " />
      <TextBlock Text="color" Foreground="Blue" />
      <TextBlock Text=" " />
      <TextBlock Text="button" Foreground="DarkOrange" />
    </WrapPanel>
  </Button.Content>
</Button>
```

Content property dozvoljava samo jedan child element pa je bilo potrebno obuhvatiti child komeponente u WrapPanel. Paneli, kao što je WrapPanel, igraju veoma važnu ulogu u Avalonia i predstavljaju kontejnere za druge kontrole.

Isti rezultat se može postići na sledeći način. Ovaj način je jednostavniji i češće je u upotrebi.

```
<Button>
  <WrapPanel>
    <TextBlock Text="A" Foreground="Red" />
    <TextBlock Text=" " />
    <TextBlock Text="multi" Foreground="Green" />
    <TextBlock Text=" " />
    <TextBlock Text="color" Foreground="Blue" />
    <TextBlock Text=" " />
    <TextBlock Text="button" Foreground="DarkOrange" />
```

```
  </WrapPanel>
</Button>
```

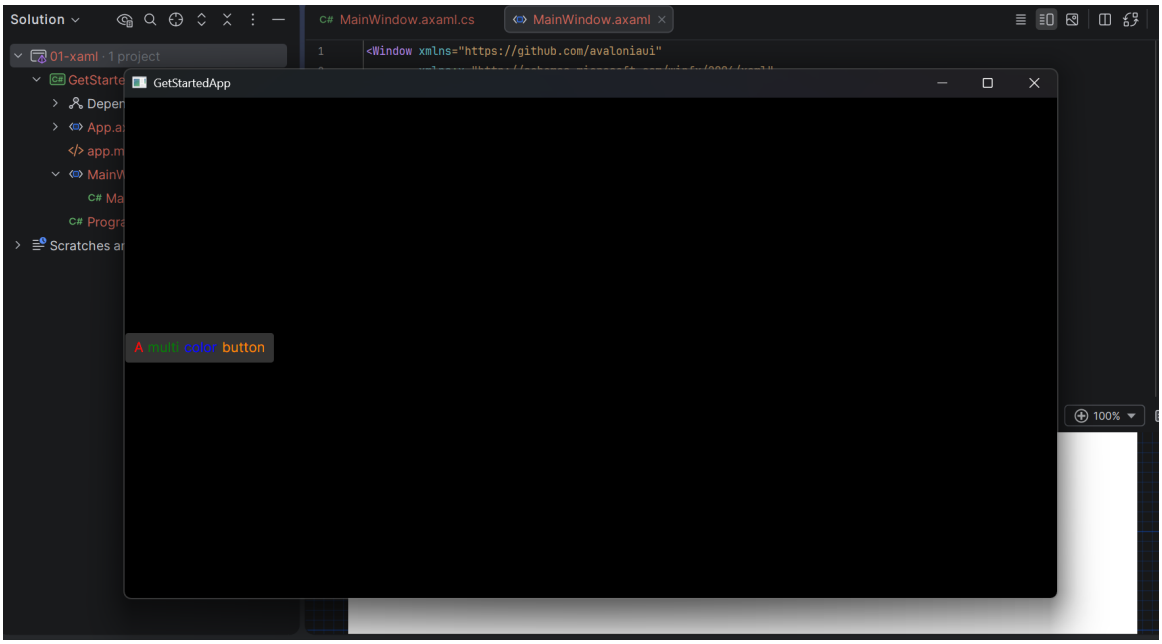

Figure 9: Rezultat 3: Dugme sa višebojnim sadržajem (isti vizuelni rezultat za obje prikazane varijante)

3 Code-behind

Mini-projekat: 02-events

Pre nego što krenete sa sekcijama *Code-behind* i *Događaji* napraviti novi projekat:

```
dotnet new avalonia.app -o 02-events -n GetStartedApp
```

U Rider-u otvorite folder `02-events` (File/Open) i pokrenite aplikaciju.

Fajlovi koje menjate: `MainWindow.axaml`, `MainWindow.axaml.cs`

Osnovna odlika svake aplikacije je da implementira funkcionalnosti, uključujući obradu događaja (eng. handling events) kao što je korisnički klik. U Avalonia, ponašanje nakon okidanja događaja se može implementirati u kodu koji je povezan sa markup-om (XAML). Ovaj tip koda se naziva code-behind.

Primer prikazuje XAML kod sa event handler-om, a zatim code-behind koji odgovara tom kodu.

```
<!-- MainWindow.axaml (zamenite ceo fajl) -->
<Window xmlns="https://github.com/avaloniaui"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        x:Class="GetStartedApp.MainWindow"
        Title="Events" Width="420" Height="220">
  <StackPanel Margin="12" Spacing="8">
    <Button Content="Klikni me" Click="Button_Click" />
  </StackPanel>
</Window>
```

```
// MainWindow.axaml.cs (zamenite ceo fajl)
using Avalonia.Controls;
using Avalonia.Interactivity;

namespace GetStartedApp;

public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();
    }

    private void Button_Click(object? sender, RoutedEventArgs e)
    {
        Title = "Kliknuto!";
    }
}
```

InitializeComponent metoda se poziva iz code-behind-a, iz konstruktora klase. Ova metoda zadužena je da spoji (eng. merge) UI koji se definiše u markup kodu sa code-behind klasom.

Napomena: InitializeComponent se generiše za vas kada se kreira aplikacija i ne treba je implementirati ručno.

4 Događaji i obrada događaja (eng. Events and handling events)

Napomena: nastavljate u mini-projektu `02-events`.

Većina UI framework-a su event driven. Sve kontrole, uključujući `Window` kontrolu, izlažu događaje na koje se može odreagovati. Možete se prijaviti (eng. subscribe) na ove događaje, što znači da će vaša aplikacija biti obaveštena kada se ovaj događaj desi i tada je moguće odreagovati na određeni način. Reakcija na događaj se zapravo definiše u event handler-u.

Postoji više tipova događaja ali oni koji se najčešće koriste su vezani za interakciju sa mišem ili tastaturom. U Avalonia se često koriste događaji kao što su: `KeyDown`, `KeyUp`, `PointerPressed`, `PointerReleased`, `PointerEntered`, `PointerExited`.

4.1 Dodavanje event handler-a u Avalonia (Rider)

1) Direktno u XAML-u

U XAML-u dodajte handler kao atribut kontrole (npr. dugmeta). Ako metoda ne postoji, Rider obično nudi quick-fix (**Alt+Enter**) da se kreira metoda u odgovarajućem code-behind fajlu.

```
<!-- MainWindow.axaml -->
<Button Content="Klik" Click="Button_Click" />
```

```
// MainWindow.axaml.cs
using Avalonia.Interactivity;

private void Button_Click(object? sender, RoutedEventArgs e)
{
    // reakcija na klik
}
```

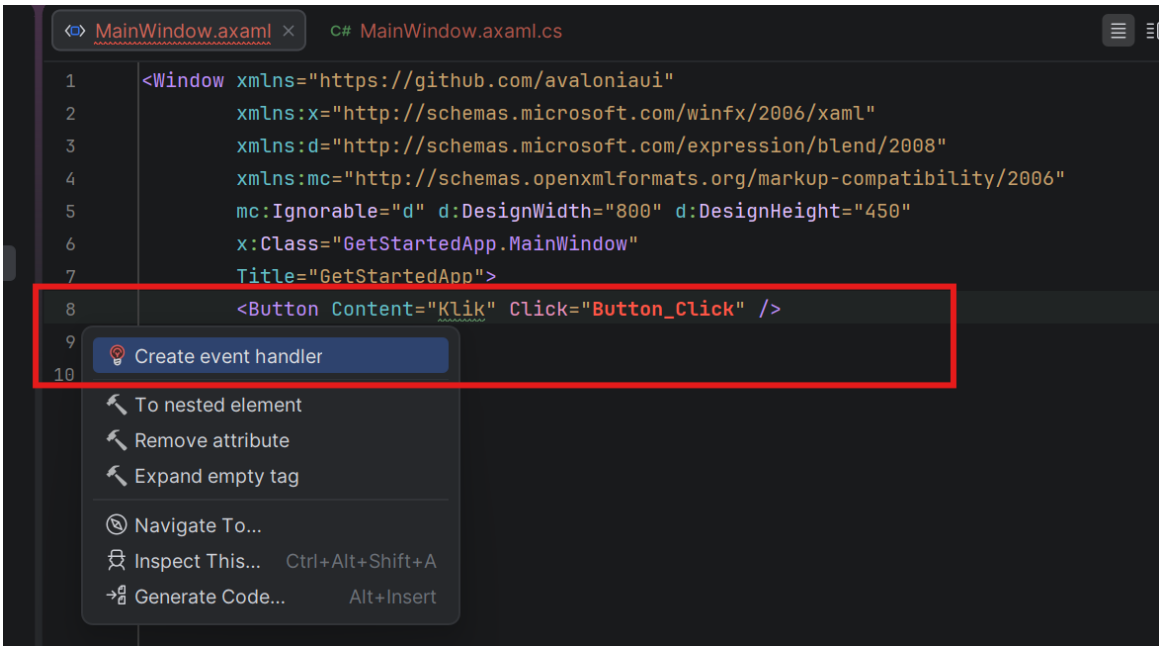

Figure 10: Slika 14. Dodavanje događaja kao XAML atribut (Rider)

2) Pretplata u code-behind-u

Drugi način je da se pretplatite na događaj u konstruktoru (posle `InitializeComponent()`). U tom slučaju najčešće je potrebno pronaći kontrolu po imenu (`Name` / `x:Name`).

```
// MainWindow.axaml
<Button x:Name="MyButton" Content="Klik" />
```

```
// MainWindow.axaml.cs
using Avalonia.Controls;
using Avalonia.Interactivity;

public MainWindow()
{
    InitializeComponent();
    var button = this.FindControl<Button>("MyButton");
    button.Click += Button_Click;
}
```

```
MainWindow.axaml        C# MainWindow.axaml.cs ×
1   using Avalonia.Controls;
2   using Avalonia.Interactivity;
3
4   namespace GetStartedApp;
5
    1 usage
6   public partial class MainWindow : Window
7   {
        1 usage
8       public MainWindow()
9       {
10          InitializeComponent();
11
12          var button = this.FindControl<Button>( name: "MyButton");
13          button.Click += Button_Click;
14      }
15
        1 usage
16      private void Button_Click(object? sender, RoutedEventArgs e)
17      {
18          // Ovdje ide kod koji se izvršava kada se klikne dugme.
19          // Za potrebe primjera, možemo samo promijeniti naslov prozora:
20          Title = "Kliknuto!";
21      }
22  }
```

Figure 11: Slika 15. Pretplata na događaj u code-behind-u (Rider)

Napomena: U praksi se handler-i najčešće dodaju kroz XAML ili kod; nije potrebno oslanjati se na poseban "Properties" panel.

5 Paneli (eng. Panels)

Mini-projekat: 03-layouts

Napravite novi projekat:

```
dotnet new avalonia.app -o 03-layouts -n GetStartedApp
```

Fajl koji menjate: `MainWindow.axaml`

Kako radite podprimere: za svaki panel primer zamenite ceo `MainWindow.axaml` odgovarajućim listingom, pokrenite aplikaciju i posmatrajte raspored.

Panel, kao jedna od najvažnijih kontrola, predstavlja kontejner za druge kontrole i upravlja sa rasporedom istih na prozoru. S obzirom da Window kontrola može da ima samo jednu child kontrolu, obično se koristi panel da podeli prostor na regije gde svaka regija može ponovo biti panel.

U nastavku će biti navedeni i objašnjeni neki od panela koji se koriste u Avalonia.

5.1 Wrap panel

WrapPanel pozicionira svaku od svojih child kontrola jednu do druge, horizontalno (default varijanta) ili vertikalno, dokle god ima dovoljno prostora. Kada ne postoji dovoljno prostora, child elementi se wrap-uju u novi red i nastavlja se njihovo ređanje.

WrapPanel treba koristiti kada imamo listu kontrola koje želimo da rasporedimo vertikalno ili horizontalno.

Kada WrapPanel koristi horizontalnu orijentaciju, child kontrole će imati istu visinu, zasnovanu na visini najvišeg elementa. Kada je WrapPanel vertikalno orijentisan, child kontrole će imati istu širinu kao najširi element.

Primer XAML koda (Avalonia) za WrapPanel:

```
<!-- MainWindow.axaml -->
<Window xmlns="https://github.com/avaloniaui"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        x:Class="GetStartedApp.MainWindow"
        Title="WrapPanel primer" Width="520" Height="260">
  <WrapPanel Margin="12">
    <Button Content="JEDAN" Width="120" Height="40" Margin="6" />
    <Button Content="DVA" Width="120" Height="40" Margin="6" />
    <Button Content="TRI" Width="120" Height="40" Margin="6" />
    <Button Content="CETIRI" Width="120" Height="40" Margin="6" />
    <Button Content="PET" Width="120" Height="40" Margin="6" />
    <Button Content="SEST" Width="120" Height="40" Margin="6" />
    <Button Content="SEDAM" Width="120" Height="40" Margin="6" />
  </WrapPanel>
</Window>
```

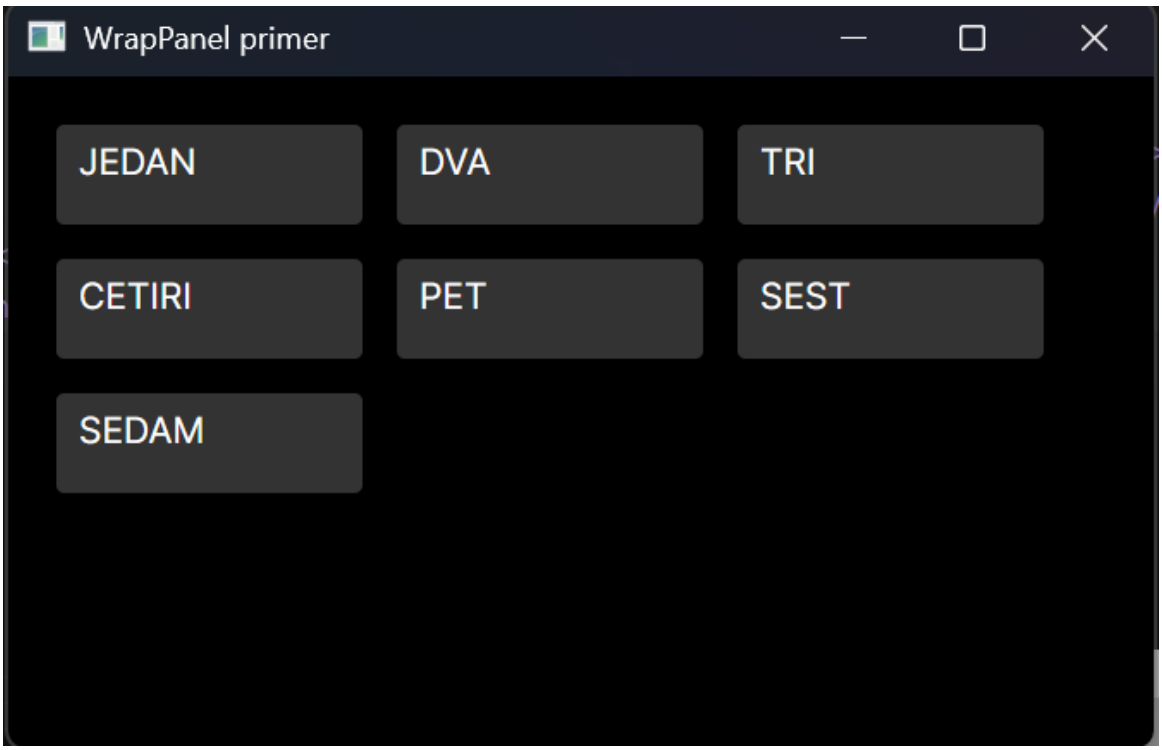

Figure 12: Rezultat 4. Horizontalni raspored elemenata u WrapPanel panelu

Očekivano: dugmad se ređaju levo–desno i "prelamaju" u novi red kada nema dovoljno širine.

5.2 Stack panel

StackPanel raspoređuje svoje child kontrole jednu za drugom, u jednom smeru. Podrazumevana orijentacija je vertikalna, što znači da se elementi ređaju odozgo nadole.

Za razliku od WrapPanel-a, StackPanel ne prelama sadržaj u novi red ili kolonu. Elementi se jednostavno slažu redom, u skladu sa zadatom orijentacijom.

U sledećem primeru koristi se vertikalni `StackPanel` koji sadrži više pravougaonika različitih visina. Na ovaj način se jasno vidi kako StackPanel raspoređuje elemente jedan ispod drugog.

Primer XAML koda (Avalonia) za StackPanel:

```
<!-- MainWindow.axaml -->
<Window xmlns="https://github.com/avaloniaui"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        x:Class="GetStartedApp.MainWindow"
        Title="StackPanel primer" Width="420" Height="260">
  <StackPanel Width="200">
    <Rectangle Fill="Red" Height="50"/>
    <Rectangle Fill="Blue" Height="50"/>
    <Rectangle Fill="Green" Height="100"/>
    <Rectangle Fill="Orange" Height="50"/>
  </StackPanel>
</Window>
```

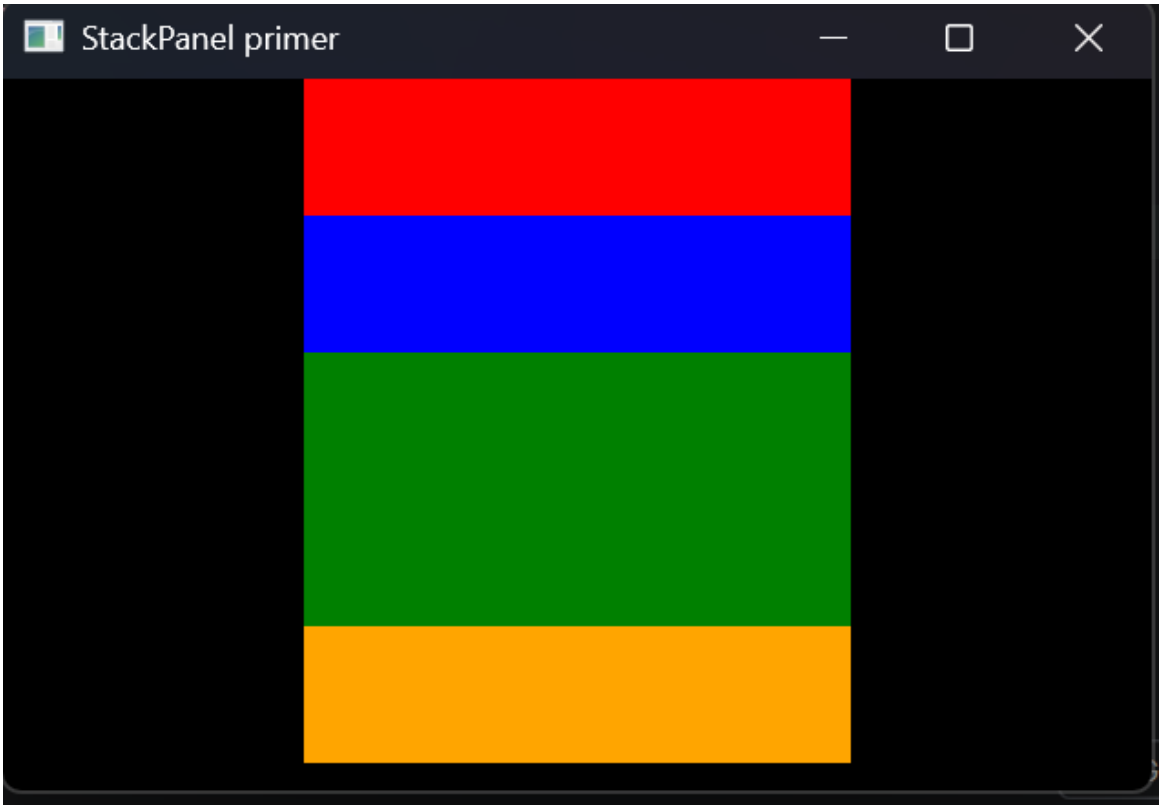

Figure 13: Rezultat 5. Vertikalni raspored elemenata u StackPanel panelu

U ovom primeru svi pravougaonici imaju istu širinu, dok im je visina definisana pojedinačno. Pošto je orijentacija StackPanel-a vertikalna, elementi se raspoređuju jedan ispod drugog, bez prelamanja sadržaja.

Očekivano: pravougaonici se prikazuju u jednoj koloni, odozgo nadole, pri čemu svaki element zadržava svoju zadatu visinu.

5.3 Grid panel

Grid panel predstavlja jedan od najmoćnijih layout panela u Avalonia framework-u. Omogućava raspoređivanje kontrola u mreži koja se sastoji od redova i kolona.

Kontrole unutar Grid-a pozicioniraju se pomoću atributa `Grid.Row` i `Grid.Column`, koji određuju u kom redu i koloni će se kontrola nalaziti. Pored toga, kontrola se može proširiti preko više redova pomoću atributa `Grid.RowSpan`.

Redovi i kolone mogu se definisati na dva načina:

- **dužom sintaksom**, pomoću `<Grid.RowDefinitions>` i `<Grid.ColumnDefinitions>` blokova
- **skraćenom sintaksom**, navođenjem vrednosti direktno u atributima `RowDefinitions` i `ColumnDefinitions`

Veličine redova i kolona mogu biti zadate na više načina:

- **fiksnom vrednošću** (npr. `100`) – širina ili visina izražena u pikselima
- **Auto** – veličina se prilagođava sadržaju
- **zvezdica (`*`)** – preostali prostor se deli proporcionalno

Na primer, definicija `100,1.5*,4*` znači da prva kolona ima fiksnu širinu od 100 piksela, a da se preostali prostor deli između druge i treće kolone u odnosu 1.5:4.

Duža sintaksa je korisna kada želimo eksplicitno da prikažemo strukturu Grid-a:

```
<Grid Margin="4">
  <Grid.RowDefinitions>
    <RowDefinition Height="Auto"/>
    <RowDefinition Height="Auto"/>
    <RowDefinition Height="Auto"/>
  </Grid.RowDefinitions>

  <Grid.ColumnDefinitions>
    <ColumnDefinition Width="100"/>
    <ColumnDefinition Width="1.5*"/>
    <ColumnDefinition Width="4*"/>
  </Grid.ColumnDefinitions>
</Grid>
```

U praksi se u Avalonia često koristi **skraćena sintaksa**, jer je kraća i preglednija.

Primer XAML koda (Avalonia) za Grid:

```
<!-- MainWindow.axaml -->
<Window xmlns="https://github.com/avaloniaui"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        x:Class="GetStartedApp.MainWindow"
        Title="Grid primer"
        Width="520"
        Height="220">
  <Grid ColumnDefinitions="100,1.5*,4*"
        RowDefinitions="Auto,Auto,Auto"
        Margin="4">
    <TextBlock Text="Col0Row0:" Grid.Row="0" Grid.Column="0"/>
    <TextBlock Text="Col0Row1:" Grid.Row="1" Grid.Column="0"/>
    <TextBlock Text="Col0Row2:" Grid.Row="2" Grid.Column="0"/>

    <CheckBox Content="Col2Row0"
              Grid.Row="0"
```

```
                Grid.Column="2"/>

    <Button Content="SpansCol1-2Row1-2"
            Grid.Row="1"
            Grid.Column="1"
            Grid.RowSpan="2"/>
  </Grid>
</Window>
```

U ovom primeru Grid ima tri reda i tri kolone. Prva kolona ima fiksnu širinu od 100 piksela, dok druga i treća kolona proporcionalno dele preostali prostor.

Tekstualne kontrole su postavljene u prvu kolonu, a `CheckBox` je smešten u prvi red treće kolone. Dugme koristi atribut `Grid.RowSpan`, pa zauzima više redova – počevši od ćelije u redu 1 i koloni 1, šireći se jedan red nadole.

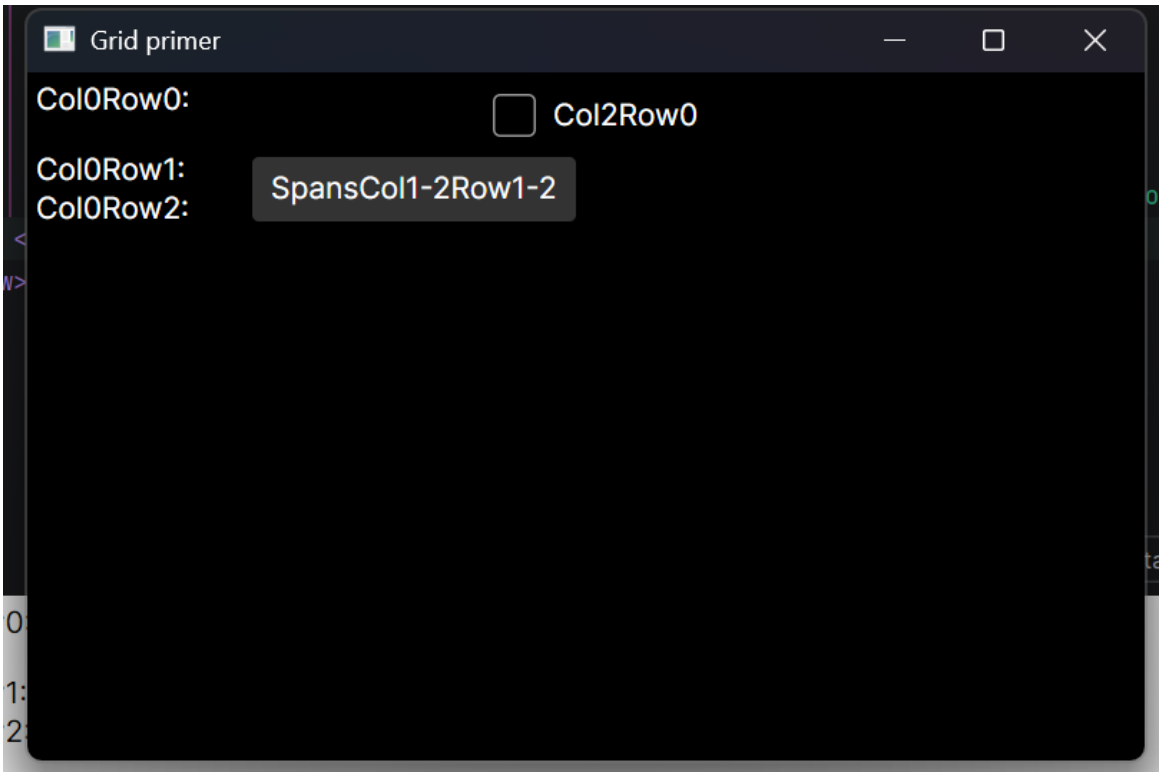

Figure 14: Rezultat 6. Raspored kontrola u Grid panelu uz korišćenje redova, kolona i atributa `Grid.RowSpan`

Očekivano: kontrole se raspoređuju u mrežu definisanu redovima i kolonama, dok dugme zauzima veći prostor po visini jer se prostire preko dva reda.

5.4 Dock panel

DockPanel je layout panel koji raspoređuje svoje child kontrole uz ivice kontejnera. Svaka kontrola može biti "prikačena" (eng. docked) uz jednu od sledećih ivica:

- **Top** – gornja ivica
- **Bottom** – donja ivica
- **Left** – leva ivica
- **Right** – desna ivica

Pozicija kontrole određuje se pomoću atributa `DockPanel.Dock`. Kontrole se raspoređuju redosledom kojim su definisane u XAML kodu. Kada se postavi nova kontrola, DockPanel uzima u obzir prostor koji su već zauzele prethodne kontrole, pa se elementi nikada ne preklapaju.

Važna karakteristika DockPanel-a je da **poslednja kontrola popunjava sav preostali prostor**. Zbog toga je potrebno da poslednji element nema definisan `DockPanel.Dock` atribut.

Kada je kontrola dock-ovana uz određenu ivicu, potrebno je definisati dimenziju koja je **normalna (perpendicular)** na tu ivicu. Na primer:

- kontrola dock-ovana uz `Top` ili `Bottom` treba da ima definisanu `Height`
- kontrola dock-ovana uz `Left` ili `Right` treba da ima definisanu `Width`

Primer XAML koda (Avalonia) za DockPanel:

```
<!-- MainWindow.axaml -->
<Window xmlns="https://github.com/avaloniaui"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        x:Class="GetStartedApp.MainWindow"
        Title="DockPanel primer">

  <DockPanel Width="300" Height="300">

    <Rectangle Fill="Red"
               Height="100"
               DockPanel.Dock="Top"/>

    <Rectangle Fill="Blue"
               Width="100"
               DockPanel.Dock="Left"/>

    <Rectangle Fill="Green"
               Height="100"
               DockPanel.Dock="Bottom"/>

    <Rectangle Fill="Orange"
               Width="100"
               DockPanel.Dock="Right"
               Opacity="0.5"/>

    <Rectangle Fill="Gray"/>

  </DockPanel>

</Window>
```

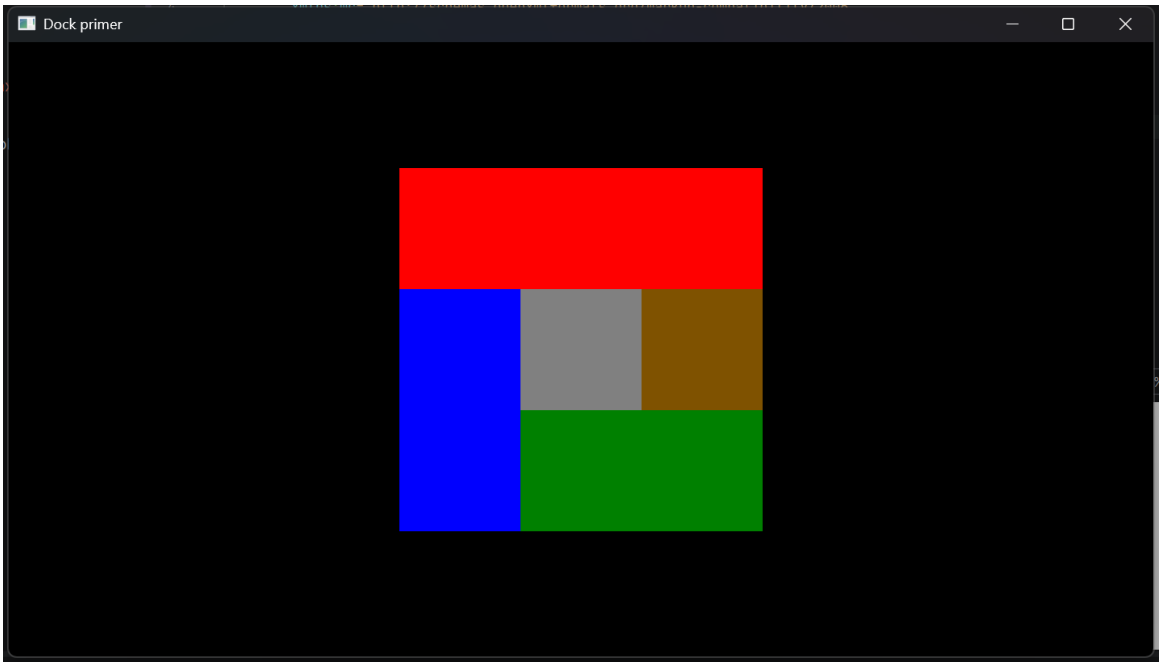

Figure 15: Rezultat 7. Raspored elemenata u DockPanel panelu

U ovom primeru:

- crveni pravougaonik je dock-ovan uz gornju ivicu
- plavi pravougaonik je dock-ovan uz levu ivicu
- zeleni pravougaonik je dock-ovan uz donju ivicu
- narandžasti pravougaonik je dock-ovan uz desnu ivicu

Poslednji element (sivi pravougaonik) nema definisan `DockPanel.Dock` atribut i zato automatski popunjava sav preostali prostor u panelu.

Narandžasti pravougaonik ima postavljen atribut `Opacity="0.5"` kako bi se lakše videlo da se elementi ne preklapaju.

Očekivano: kontrole se raspoređuju uz ivice DockPanel-a, dok poslednja kontrola popunjava preostali centralni prostor.

5.5 Primer kombinacije više panela – Complex Layout

U realnim aplikacijama korisnički interfejs se retko sastoji od jednog panela. Najčešće je potrebno kombinovati više različitih layout panela kako bi se dobio pregledan raspored elemenata u prozoru.

U sledećem primeru koristi se `DockPanel` kao osnovni panel koji deli prozor na nekoliko glavnih sekcija:

- zaglavlje na vrhu prozora
- statusna traka na dnu
- bočni meni sa leve strane
- centralni deo za prikaz sadržaja

Unutar bočnog menija i centralnog dela koristi se `StackPanel` za vertikalno raspoređivanje kontrola. Na taj način se različiti paneli kombinuju kako bi se dobio složeniji raspored elemenata u interfejsu.

Primer XAML koda (Avalonia) za kombinaciju više panela:

```xml
<!-- MainWindow.axaml -->
<Window xmlns="https://github.com/avaloniaui"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        x:Class="GetStartedApp.MainWindow"
        Title="Complex layout primer"
        Width="760"
        Height="420">

  <DockPanel>

    <Border DockPanel.Dock="Top" Background="PaleVioletRed" Padding="12">
      <TextBlock Text="Uvod u Avalonia Layout-e"
                 FontSize="20"
                 FontWeight="Bold"/>
    </Border>

    <Border DockPanel.Dock="Bottom" Background="PaleVioletRed" Padding="10">
      <TextBlock Text="Status: aplikacija je pokrenuta"/>
    </Border>

    <StackPanel DockPanel.Dock="Left"
                Width="180"
                Spacing="8"
                Margin="12">
      <Button Content="Početna" Height="36"/>
      <Button Content="Paneli" Height="36"/>
      <Button Content="Binding" Height="36"/>
    </StackPanel>

    <Border Margin="12"
            Padding="16"
            Background="DeepSkyBlue">
      <StackPanel Spacing="10">
        <TextBlock Text="Glavni sadržaj"
                   FontSize="18"
                   FontWeight="Bold"/>

        <TextBlock Text="Ovaj primer prikazuje kombinaciju više layout panela u jednom
        ↪  prozoru."
                   TextWrapping="Wrap"/>

        <TextBlock Text="DockPanel raspoređuje zaglavlje, bočni meni i statusnu traku,
        ↪  dok se StackPanel koristi za grupisanje dugmadi i sadržaja."
                   TextWrapping="Wrap"/>

        <Button Content="Akcija"
                Width="120"
                HorizontalAlignment="Left"/>
      </StackPanel>
    </Border>

  </DockPanel>
</Window>
```

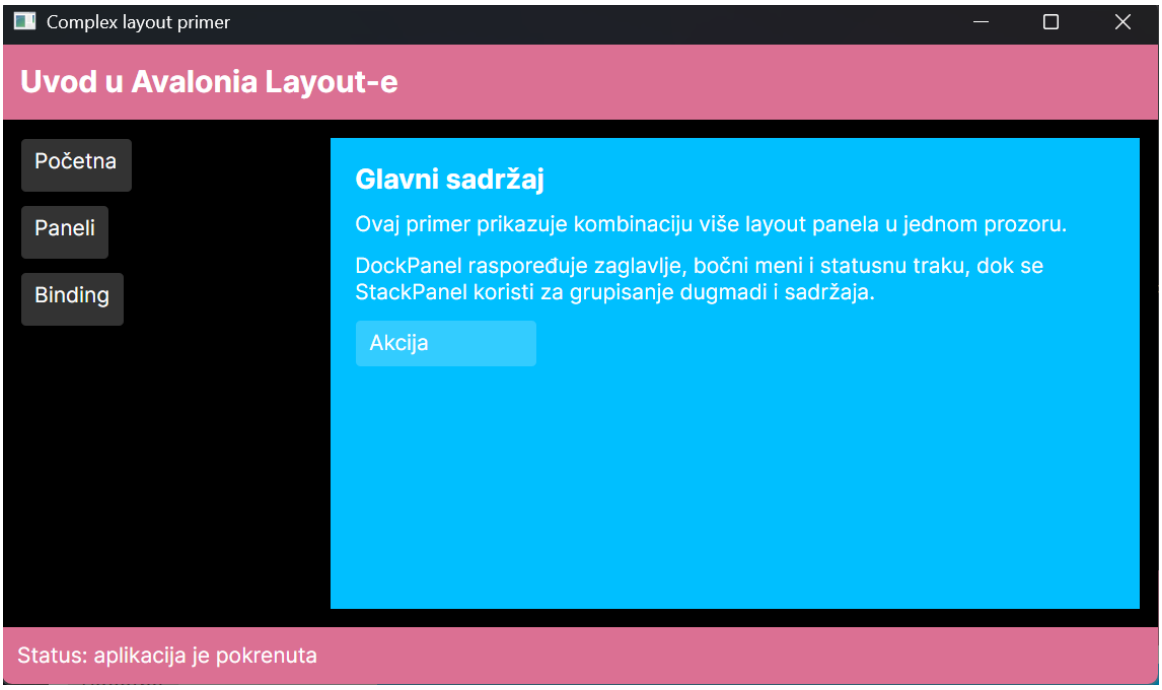

Figure 16: Rezultat 8. Primer kombinacije više layout panela u jednom prozoru

U ovom primeru:

- `DockPanel` raspoređuje glavne sekcije prozora
- zaglavlje i statusna traka su dock-ovani uz gornju i donju ivicu
- `StackPanel` na levoj strani predstavlja jednostavan meni
- centralni deo prikazuje sadržaj aplikacije

Očekivano: zaglavlje se prikazuje na vrhu prozora, statusna traka na dnu, meni sa leve strane, dok centralni deo popunjava preostali prostor i prikazuje glavni sadržaj aplikacije.

6 Data binding

Mini-projekat: 04-binding

Napravite novi projekat:

```
dotnet new avalonia.app -o 04-binding -n GetStartedApp
```

Fajlovi koje menjate: `MainWindow.axaml`, `MainWindow.axaml.cs` + novi fajlovi iz ove sekcije.
Kako radite podprimere:

- prvo dodajte `BindingDemoViewModel.cs` i ažurirajte `MainWindow.axaml.cs` (podsekcija "Priprema")
- zatim u svakoj podsekciji zamenite `MainWindow.axaml` prikazanim XAML-om i pokrenite aplikaciju

Većina aplikacija omogućava korisniku da manipuliše sa nekim podacima. Kada su podaci učitani u aplikaciju (npr. iz neke baze podataka) potrebno ih je prikazati korisniku. Osim toga, korisniku se često omogućava da menja te podatke pa se mora voditi računa da bude tačno stanje podataka u bazi.

Da bi olakšala prikaz i interakciju sa podacima, Avalonia koristi data binding. Data binding predstavlja mehanizam koji omogućava komunikaciju UI elemenata sa podacima. Kada je binding uspostavljen i kada se podaci promene, moguće je automatski promeniti i UI elemente koji zavise od tih podataka. Obrnuto je takođe moguće.

Moguće je vršiti binding između dve kontrole kao i binding na resurs.

Da bi se mogle detektovati promene na source-u (primenjivo na OneWay i TwoWay binding), source mora implementirati odgovarajući mehanizam notifikacije promena, kao što je `INotifyPropertyChanged`.

Na primeru biće objašnjeni različiti načini upotrebe binding-a. U nastavku koristimo jednostavan ViewModel kao `DataContext`.

6.1 Priprema: DataContext (primer)

Napravite klasu `BindingDemoViewModel.cs` i postavite je kao `DataContext` u `MainWindow.axaml.cs`.

```
// BindingDemoViewModel.cs
using System.Collections.ObjectModel;
using System.ComponentModel;
using System.Runtime.CompilerServices;

namespace GetStartedApp;

public class BindingDemoViewModel : INotifyPropertyChanged
{
    private string _test1 = "Pocetna vrednost";
    public string Test1
    {
        get => _test1;
        set
        {
            if (_test1 == value) return;
            _test1 = value;
            OnPropertyChanged();
        }
    }

    private string _test2 = "Edituj me";
    public string Test2
    {
        get => _test2;
        set
        {
            if (_test2 == value) return;
            _test2 = value;
            OnPropertyChanged();
        }
    }

    private string _onlySource = "";
    public string OnlySource
    {
        get => _onlySource;
        set
        {
            if (_onlySource == value) return;
            _onlySource = value;
            OnPropertyChanged();
        }
    }
```

```
    private string _immediateText = "";
    public string ImmediateText
    {
        get => _immediateText;
        set
        {
            if (_immediateText == value) return;
            _immediateText = value;
            OnPropertyChanged();
        }
    }

    private string _lostFocusText = "";
    public string LostFocusText
    {
        get => _lostFocusText;
        set
        {
            if (_lostFocusText == value) return;
            _lostFocusText = value;
            OnPropertyChanged();
        }
    }

    public ObservableCollection<string> Test3 { get; } = new()
    {
        "Audi",
        "BMW",
        "Opel",
    };

    private string? _selectedTest3;
    public string? SelectedTest3
    {
        get => _selectedTest3;
        set
        {
            if (_selectedTest3 == value) return;
            _selectedTest3 = value;
            OnPropertyChanged();
        }
    }

    private int _itemCounter = 1;
    public void AddItem() => Test3.Add($"Item {_itemCounter++}");

    public event PropertyChangedEventHandler? PropertyChanged;
    private void OnPropertyChanged([CallerMemberName] string? propertyName = null) =>
        PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(propertyName));
}
```

```
// MainWindow.axaml.cs
using Avalonia.Controls;

namespace GetStartedApp;

public partial class MainWindow : Window
```

```
{
    public BindingDemoViewModel ViewModel { get; } = new();

    public MainWindow()
    {
        InitializeComponent();
        DataContext = ViewModel;
    }
}
```

6.2 Binding na resurs – string (1)

Resursi se najčešće definišu u okviru `Window.Resources` bloka i koriste se preko `StaticResource`. Na ovaj način je moguće centralizovano definisati vrednosti koje se koriste na više mesta u interfejsu.

```
<!-- MainWindow.axaml (primer) -->
<Window xmlns="https://github.com/avaloniaui"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        x:Class="GetStartedApp.MainWindow"
        Title="Resources (string)" Width="520" Height="220">
  <Window.Resources>
    <x:String x:Key="strTestBinding">Hello from resource</x:String>
  </Window.Resources>

  <StackPanel Margin="12" Spacing="8">
    <TextBox Text="{StaticResource strTestBinding}" />
  </StackPanel>
</Window>
```

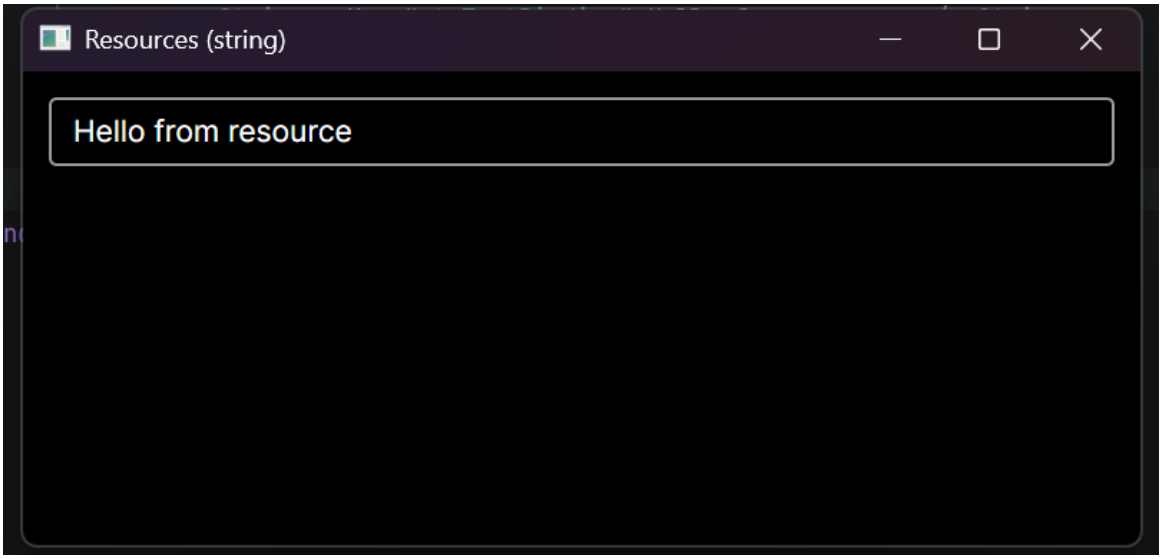

Figure 17: Rezultat 9. Binding na string resurs definisan u `Window.Resources`

U ovom primeru vrednost iz resursa `strTestBinding` automatski se prikazuje u `TextBox` kontroli. Pošto je u pitanju statički resurs, kontrola koristi unapred definisanu vrednost iz bloka `Window.Resources`.

6.3 Binding na resurs – boja (2)

Pored tekstualnih vrednosti, u resursima se često definišu i boje, četkice (brush), stilovi i drugi objekti koji se koriste u interfejsu. U sledećem primeru resurs se koristi kao brush za boju selekcije teksta.

```
<!-- MainWindow.axaml (primer) -->
<Window xmlns="https://github.com/avaloniaui"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        x:Class="GetStartedApp.MainWindow"
        Title="Resources (brush)" Width="520" Height="240">
  <Window.Resources>
    <SolidColorBrush x:Key="clrSpecialColor" Color="OrangeRed" />
  </Window.Resources>

  <StackPanel Margin="12" Spacing="8">
    <TextBox Text="Selektuj deo teksta" SelectionBrush="{StaticResource
    ↪  clrSpecialColor}" />
  </StackPanel>
</Window>
```

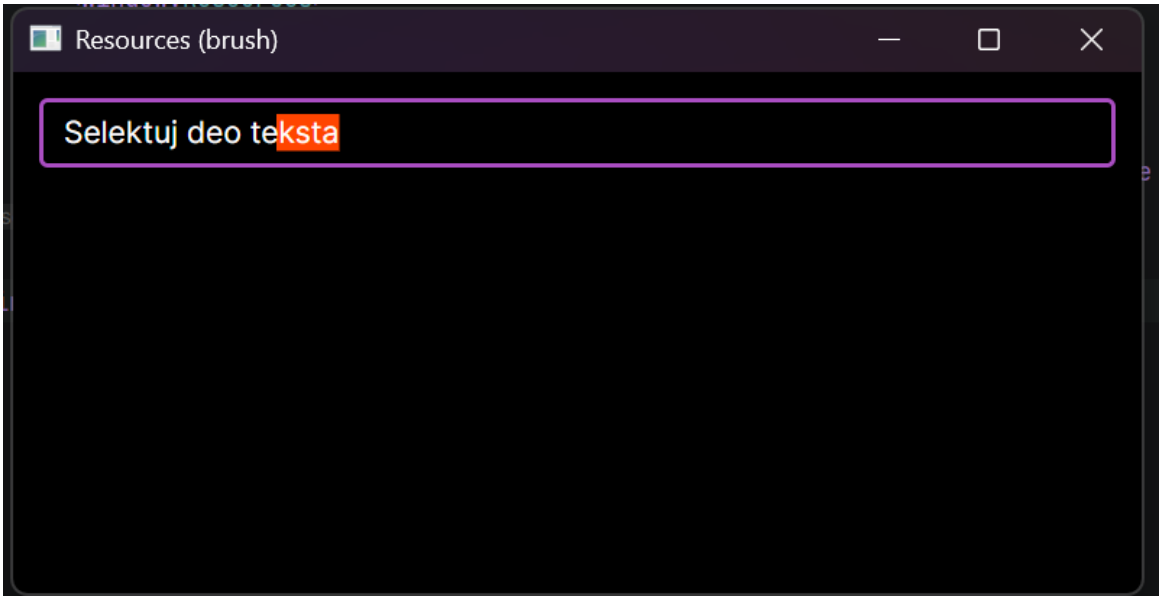

Figure 18: Rezultat 10. Korišćenje brush resursa za svojstvo `SelectionBrush`

Na slici se vidi da se pri selekciji teksta koristi boja definisana kroz resurs `clrSpecialColor`. Ovakav pristup je koristan kada želimo dosledan izgled aplikacije i jednostavnije menjanje boja na jednom mestu.

6.4 One way – ka interfejsu (7)

`OneWay` binding znači da se promene na source svojstvu automatski prenose na target kontrolu, ali izmene na samoj kontroli ne menjaju source. Ovaj tip binding-a je koristan kada želimo da interfejs samo prikazuje podatke.

```
<!-- unutar Window.Content -->
<StackPanel Margin="12" Spacing="8">
  <TextBlock Text="OneWay (Test1)" FontWeight="Bold" />
```

```
  <TextBox Text="{Binding Test1, Mode=OneWay}" />
  <Button Content="Promeni Test1 u kodu" Click="SetTest1_Click" />
</StackPanel>
```

```
// MainWindow.axaml.cs (dodati u klasu)
using System;
using Avalonia.Interactivity;

private void SetTest1_Click(object? sender, RoutedEventArgs e)
{
    ViewModel.Test1 = $"Test1: {DateTime.Now:HH:mm:ss}";
}
```

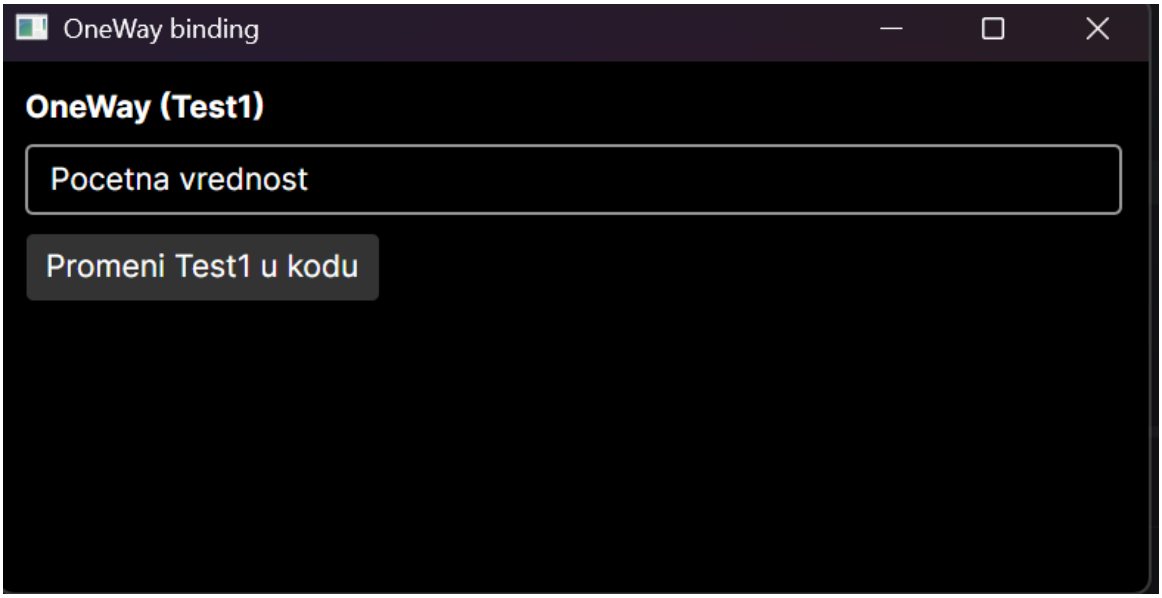

Figure 19: Rezultat 11.1. OneWay binding – početno stanje

Figure 20: Rezultat 11.2. OneWay binding – stanje nakon klika na dugme `Promeni Test1 u kodu`

Na prvoj slici prikazano je početno stanje, gde `TextBox` prikazuje početnu vrednost svojstva `Test1`. Na drugoj slici vidi se rezultat nakon izmene svojstva `Test1` u code-behind-u. Pošto je binding tipa `OneWay`, promena na source strani automatski osvežava prikaz u interfejsu.

6.5 Two way – bidirekciono (6)

`TwoWay` binding omogućava dvosmernu komunikaciju između source svojstva i kontrole. To znači da promena na source-u menja prikaz u interfejsu, ali i da korisnička izmena u kontroli menja source svojstvo. U Avalonia je `TextBox.Text` podrazumevano `TwoWay`.

```
<!-- unutar Window.Content -->
<StackPanel Margin="12" Spacing="8">
  <TextBlock Text="TwoWay (Test2)" FontWeight="Bold" />
  <TextBox Text="{Binding Test2}" />
  <TextBlock Text="{Binding Test2, StringFormat='Trenutno: {0}'}" />
</StackPanel>
```

Figure 21: Rezultat 12.1. TwoWay binding – početno stanje

Figure 22: Rezultat 12.2. TwoWay binding – stanje nakon izmene vrednosti u `TextBox` kontroli

Na prvoj slici prikazano je početno stanje svojstva `Test2`. Na drugoj slici vidi se da nakon izmene teksta u `TextBox` kontroli dolazi do ažuriranja source svojstva, a zatim se nova vrednost prikazuje i u `TextBlock` kontroli. Time se jasno vidi dvosmerna priroda `TwoWay` binding-a.

6.6 One way to source – ka kodu (8)

`OneWayToSource` binding predstavlja suprotan smer od `OneWay` binding-a. U ovom slučaju promene na kontroli ažuriraju source, ali promene na source-u ne ažuriraju kontrolu. Ovaj pristup je koristan kada korisnički unos želimo da prosledimo u model, ali ne očekujemo povratno osvežavanje iste kontrole.

```
<!-- unutar Window.Content -->
<StackPanel Margin="12" Spacing="8">
  <TextBlock Text="OneWayToSource (OnlySource)" FontWeight="Bold" />
  <TextBox Watermark="Kucaj ovde" Text="{Binding OnlySource, Mode=OneWayToSource}" />
  <TextBlock Text="{Binding OnlySource, StringFormat='Source: {0}'}" />
</StackPanel>
```

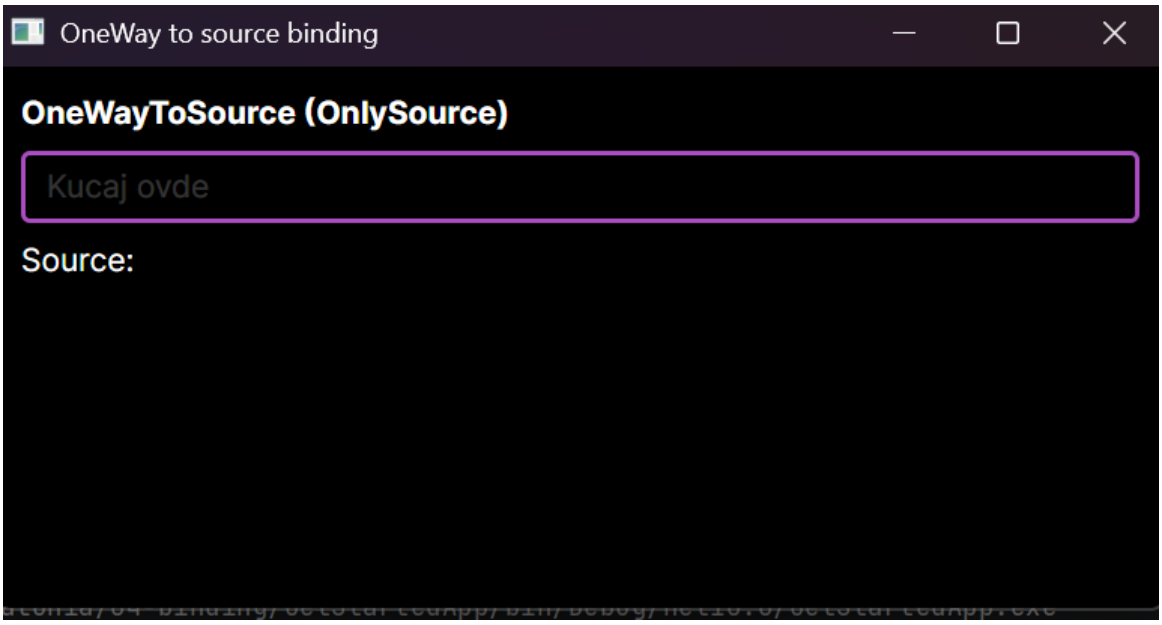

Figure 23: Rezultat 13.1. OneWayToSource binding – početno stanje

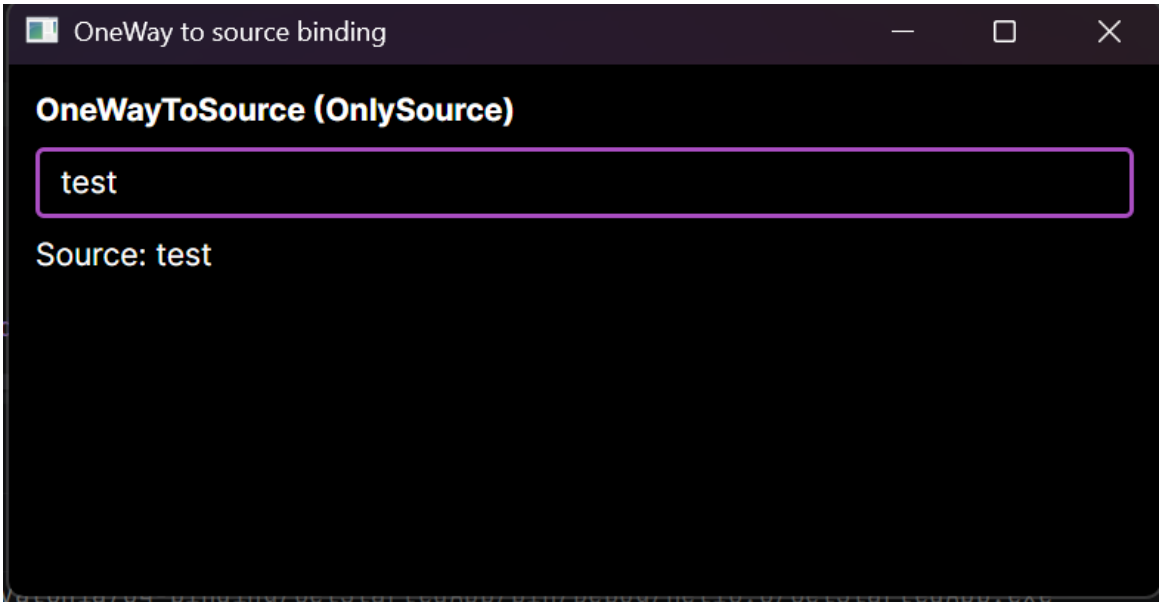

Figure 24: Rezultat 13.2. OneWayToSource binding – stanje nakon unosa teksta u kontrolu

Na prvoj slici vidi se početno stanje. Na drugoj slici korisnik unosi tekst u `TextBox`, pri čemu se vrednost prenosi u source svojstvo `OnlySource` i prikazuje u `TextBlock` kontroli. Kod `OneWayToSource` binding-a promene se prenose sa kontrole ka source-u, ali ne i obrnuto.

6.7 Binding kontrole na kontrolu (14)

Osim binding-a na `DataContext`, moguće je povezati jednu kontrolu direktno sa drugom kontrolom. U Avalonia se za to često koristi `#` sintaksa nad imenovanom kontrolom.

```
<!-- unutar Window.Content -->
<StackPanel Margin="12" Spacing="8">
  <TextBlock Text="Control-to-control binding" FontWeight="Bold" />
  <Slider x:Name="mySlider" Minimum="0" Maximum="100" Value="25" />
```

```
  <TextBlock Text="{Binding #mySlider.Value, StringFormat='Slider: {0:0}'}" />
</StackPanel>
```

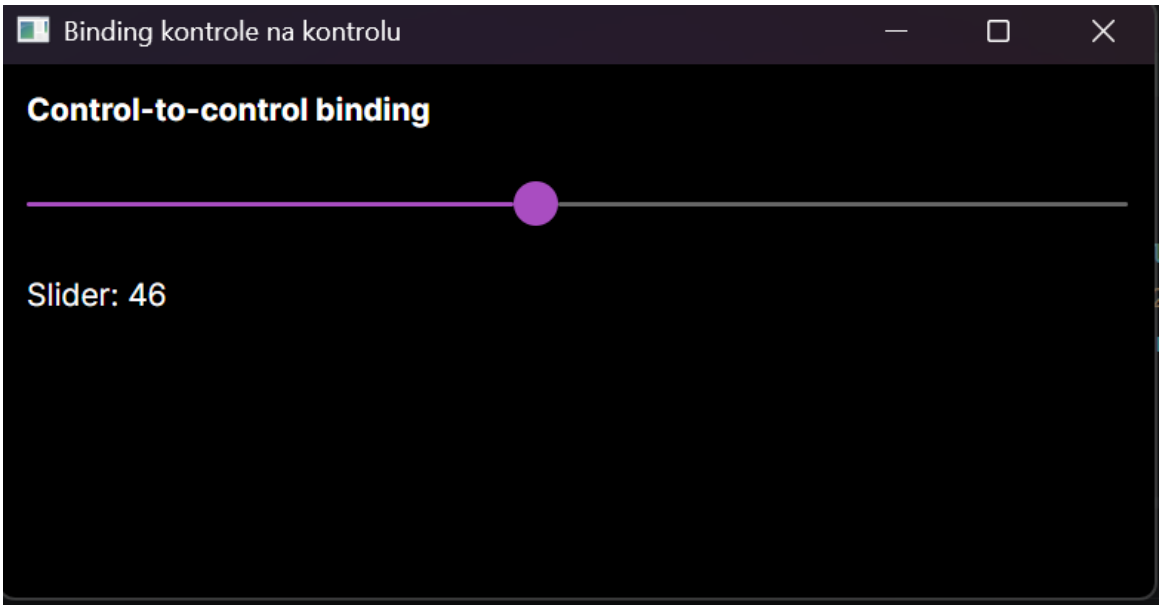

Figure 25: Rezultat 14. Binding kontrole na kontrolu korišćenjem # sintakse

U ovom primeru `TextBlock` ne čita vrednost iz `DataContext`-a, već direktno iz kontrole `Slider`. Kada se vrednost slider-a promeni, tekstualni prikaz se automatski ažurira.

6.8 Binding kontrole na kontrolu - real time (4)

`UpdateSourceTrigger` određuje kada će promena sa kontrole biti preneta na source. U Avalonia, `TextBox.Text` se podrazumevano sinhronizuje pri svakoj promeni vrednosti, odnosno ponaša se kao `PropertyChanged`. Međutim, moguće je zadati i drugačije ponašanje, kao što je ažuriranje tek nakon gubitka fokusa.

```
<!-- unutar Window.Content -->
<StackPanel Margin="12" Spacing="8">
  <TextBlock Text="UpdateSourceTrigger" FontWeight="Bold" />

  <TextBox Watermark="PropertyChanged (default)" Text="{Binding ImmediateText}" />
  <TextBlock Text="{Binding ImmediateText, StringFormat='Immediate: {0}'}" />

  <TextBox Watermark="LostFocus" Text="{Binding LostFocusText,
  ↪  UpdateSourceTrigger=LostFocus}" />
  <TextBlock Text="{Binding LostFocusText, StringFormat='LostFocus: {0}'}" />
</StackPanel>
```

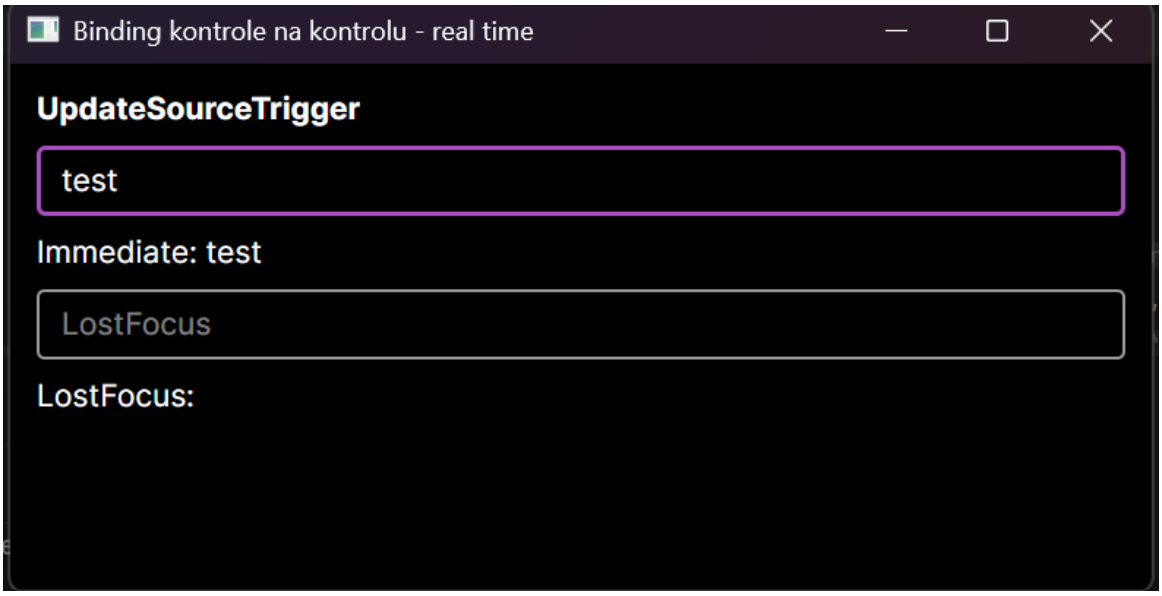

Figure 26: Rezultat 15.1. `PropertyChanged` – vrednost se ažurira odmah tokom kucanja

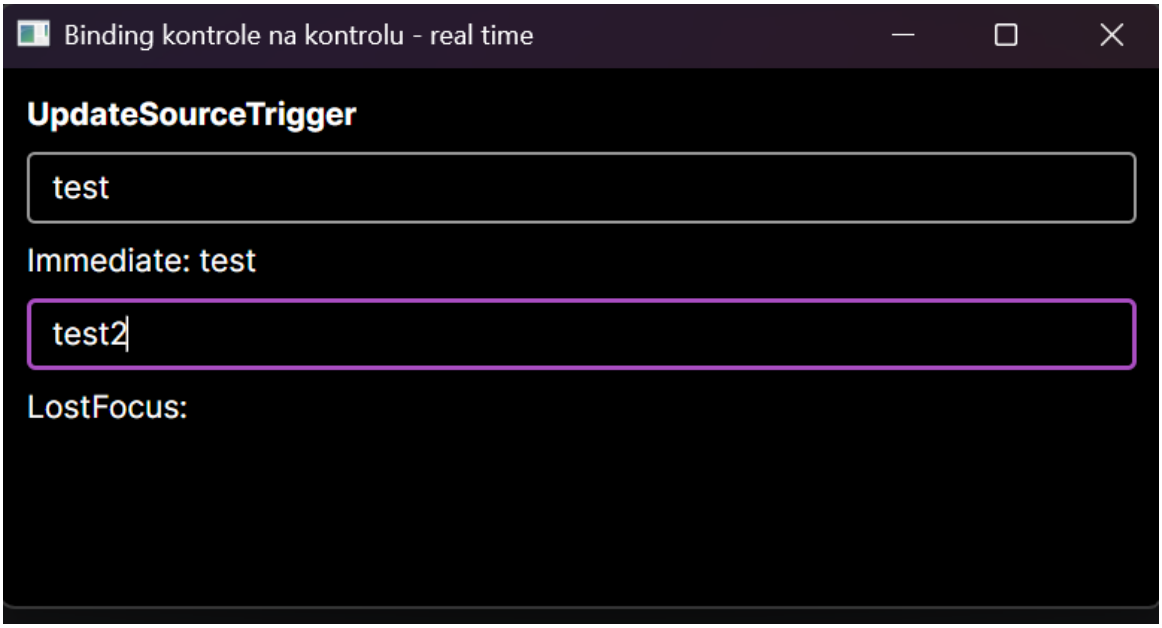

Figure 27: Rezultat 15.2. `LostFocus` – tekst je unet, ali source još nije ažuriran dok kontrola i dalje ima fokus

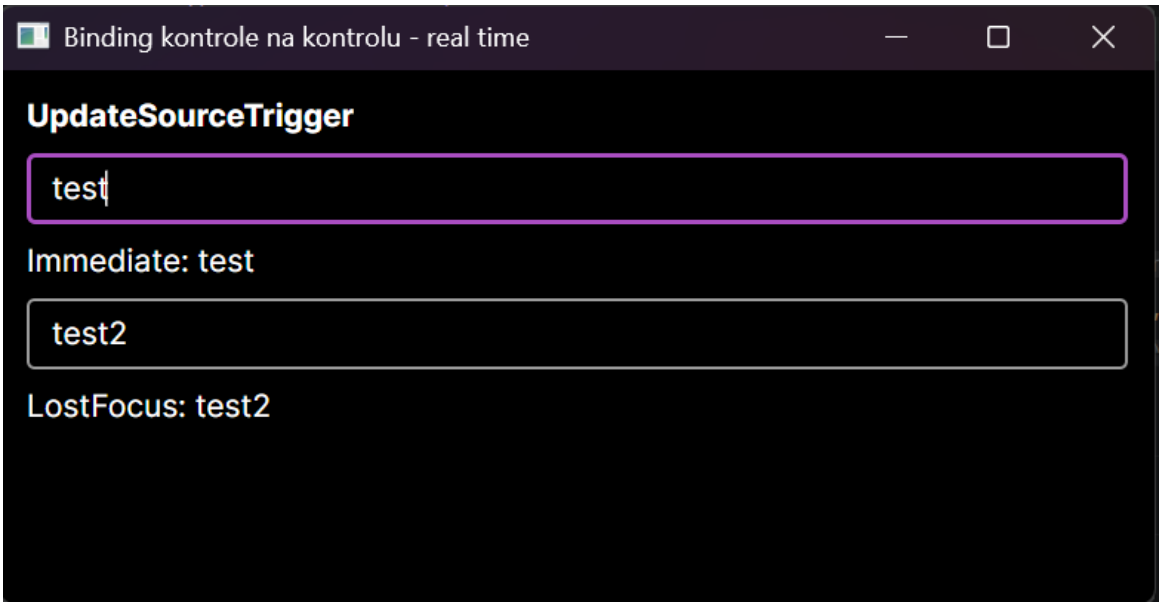

Figure 28: Rezultat 15.3. `LostFocus` – nakon gubitka fokusa source vrednost je ažurirana

Prva slika prikazuje ponašanje podrazumevanog `UpdateSourceTrigger=PropertyChanged`, gde se promena prenosi na source odmah tokom unosa. Druga slika prikazuje da kod `LostFocus` uneta vrednost još nije preneta dok je fokus i dalje na polju. Treća slika prikazuje stanje nakon gubitka fokusa, kada se source konačno ažurira.

6.9 Binding kontrole na kontrolu - real time + konverzija (5)

U nekim situacijama nije dovoljno samo preneti vrednost iz source-a u target, već je potrebno tu vrednost i transformisati. Za to se koristi `Converter`. U Avalonia se konverter najčešće implementira preko interfejsa `IValueConverter`.

```
// MultiplyConverter.cs
using System;
using System.Globalization;
using Avalonia.Data.Converters;

namespace GetStartedApp;

public class MultiplyConverter : IValueConverter
{
    public double Factor { get; set; } = 1;

    public object? Convert(object? value, Type targetType, object? parameter,
    ↪  CultureInfo culture)
    {
        if (value is double d)
            return (d * Factor).ToString("0.##", culture);
        return value?.ToString();
    }

    public object? ConvertBack(object? value, Type targetType, object? parameter,
    ↪  CultureInfo culture)
    {
        if (value is string s && double.TryParse(s, NumberStyles.Any, culture, out var
        ↪  d))
            return d / Factor;
```

```
        return value;
    }
}
```

```
<!-- MainWindow.axaml (primer) -->
<Window xmlns="https://github.com/avaloniaui"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:local="using:GetStartedApp"
        x:Class="GetStartedApp.MainWindow"
        Title="Converter" Width="520" Height="260">
  <Window.Resources>
    <local:MultiplyConverter x:Key="Mul2" Factor="2" />
  </Window.Resources>

  <StackPanel Margin="12" Spacing="8">
    <Slider x:Name="convSlider" Minimum="0" Maximum="50" Value="10" />
    <TextBlock Text="{Binding #convSlider.Value, StringFormat='Original: {0:0}'}" />
    <TextBlock Text="{Binding #convSlider.Value, Converter={StaticResource Mul2},
    ↪  StringFormat='x2: {0}'}" />
  </StackPanel>
</Window>
```

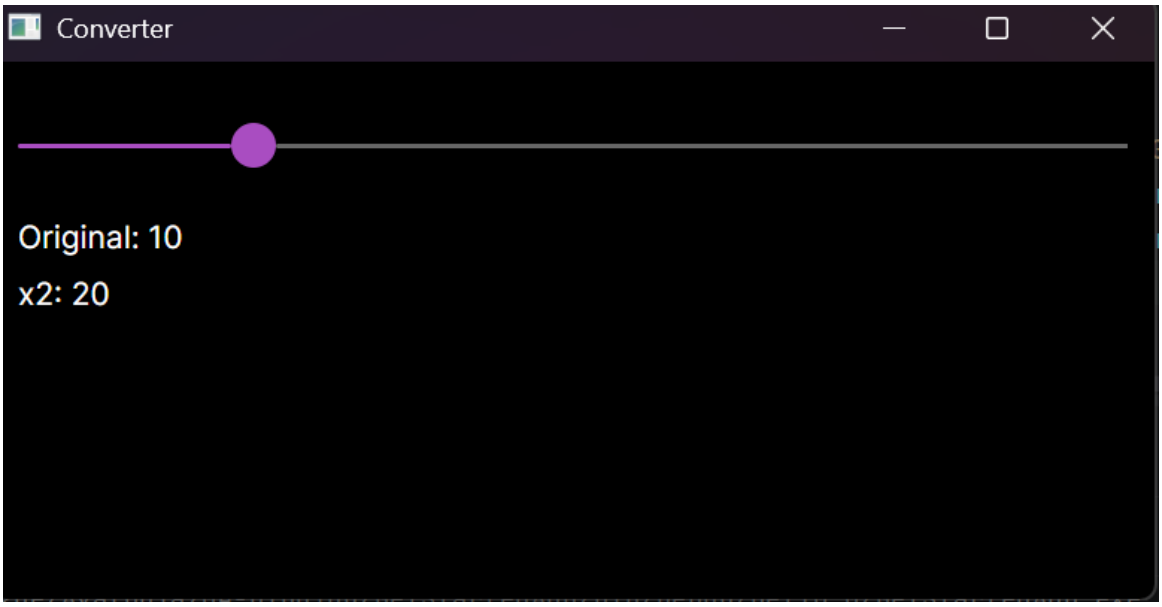

Figure 29: Rezultat 16. Primena konvertera nad vrednošću sa slider-a

Na slici se vidi originalna vrednost slider-a i rezultat nakon primene konvertera koji tu vrednost množi sa zadatim faktorom. Na taj način jedna ista izvorna vrednost može biti prikazana u različitim oblicima, u zavisnosti od potreba interfejsa.

6.10 Binding na listu (9)

Binding nije ograničen samo na pojedinačne vrednosti. Veoma čest slučaj je povezivanje UI kontrole sa kolekcijom podataka. U sledećem primeru `ComboBox` je povezan sa kolekcijom `Test3` tipa `ObservableCollection`, dok se trenutno izabrana vrednost čuva u svojstvu `SelectedTest3`.

```
<!-- unutar Window.Content -->
<StackPanel Margin="12" Spacing="8">
  <TextBlock Text="Binding na listu (Test3)" FontWeight="Bold" />

  <ComboBox ItemsSource="{Binding Test3}"
            SelectedItem="{Binding SelectedTest3}"
            Width="200" />

  <TextBlock Text="{Binding SelectedTest3, StringFormat='Selektovano: {0}'}" />

  <Button Content="Dodaj element" Click="AddItem_Click" Width="200" />
</StackPanel>
```

```
// MainWindow.axaml.cs (dodati u klasu)
using Avalonia.Interactivity;

private void AddItem_Click(object? sender, RoutedEventArgs e)
{
    ViewModel.AddItem();
}
```

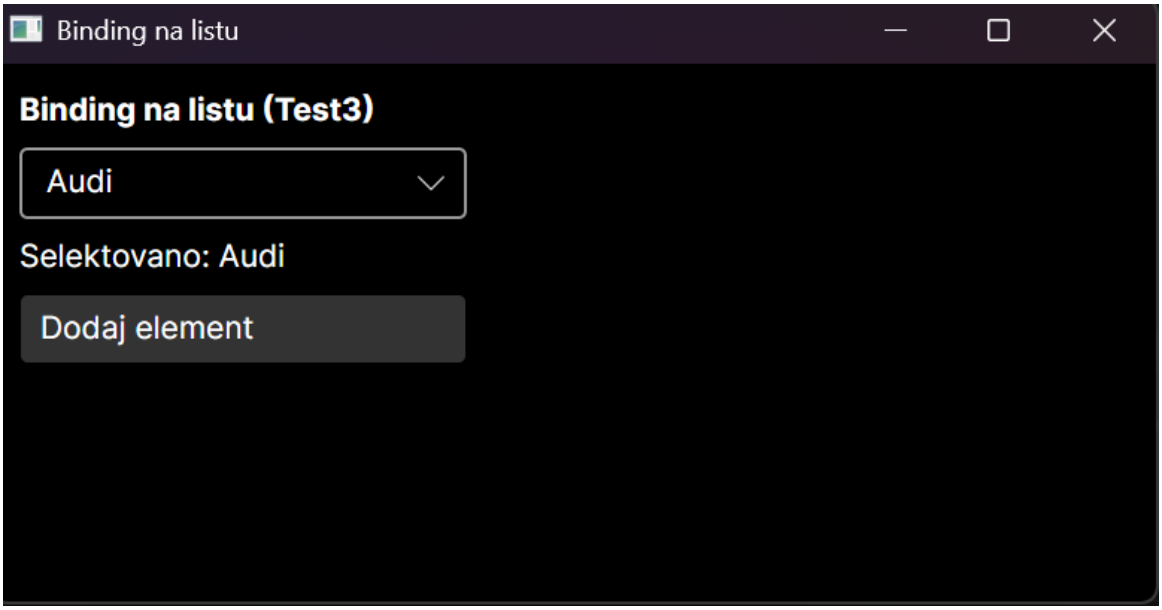

Figure 30: Rezultat 17. Binding `ComboBox` kontrole na kolekciju i prikaz selektovane vrednosti

Primer prikazuje povezivanje `ComboBox` kontrole sa kolekcijom `Test3`, kao i prikaz trenutno selektovanog elementa. Pošto je korišćena `ObservableCollection`, svaka promena kolekcije može automatski biti reflektovana u interfejsu. Dugme omogućava dinamičko dodavanje novih elemenata u listu.

DODATAK

Kontrole (Avalonia reference):

- https://docs.avaloniaui.net/docs/reference/controls/

Često korišćeni layout-i (concept):

- **Canvas:** Child controls provide their own layout.
- **DockPanel:** Child controls are aligned to the edges of the panel.
- **Grid:** Child controls are positioned by rows and columns.
- **StackPanel:** Child controls are stacked either vertically or horizontally.
- **WrapPanel:** Child controls are positioned in left-to-right order and wrapped to the next line when there are more controls on the current line than space allows.

Korisni linkovi:

- https://docs.avaloniaui.net/docs/get-started/
- https://docs.avaloniaui.net/docs/get-started/xaml-previewers
- https://docs.avaloniaui.net/docs/get-started/wpf/
- https://docs.avaloniaui.net/docs/basics/data/data-binding/
- https://docs.avaloniaui.net/docs/basics/data/data-binding/data-binding-syntax
- https://docs.avaloniaui.net/docs/guides/data-binding/binding-to-controls
- https://play.avaloniaui.net/